A framework for bilevel optimization that enables stochastic and global variance reduction algorithms

M. Dagréou[1], P. Ablin[2], S. Vaiter[3], T. Moreau[1]

[1] Université Paris-Saclay, Inria, CEA, Palaiseau, 91120, France
[2] CNRS, Université Paris-Dauphine, PSL-University, Paris, France
[3] CNRS & Université Côte d'Azur, LJAD, Nice, France

NeurIPS 2022

Problem setting

Bilevel optimization problem

$$\begin{cases} \min_{x \in \mathbb{R}^d} h(x) \triangleq F(z^*(x), x) & \text{Outer problem} \\ z^*(x) \in \arg\min_{z \in \mathbb{R}^p} G(z, x) & \text{Inner problem} \end{cases}$$

Problem setting

Bilevel optimization problem

$$\begin{cases} \min_{x\in\mathbb{R}^d} h(x) \triangleq F(z^*(x), x) & \text{Outer problem} \\ z^*(x) \in \arg\min_{z\in\mathbb{R}^p} G(z, x) & \text{Inner problem} \end{cases}$$

ERM setting:

$$F(z,x) = \frac{1}{m}\sum_{j=1}^{m} F_j(z,x), \quad G(z,x) = \frac{1}{n}\sum_{i=1}^{n} G_i(z,x)$$

Problem setting

Bilevel optimization problem

$$\begin{cases} \min_{x\in\mathbb{R}^d} h(x) \triangleq F(z^*(x), x) & \text{Outer problem} \\ z^*(x) \in \arg\min_{z\in\mathbb{R}^p} G(z, x) & \text{Inner problem} \end{cases}$$

ERM setting:

$$F(z,x) = \frac{1}{m}\sum_{j=1}^{m} F_j(z,x), \quad G(z,x) = \frac{1}{n}\sum_{i=1}^{n} G_i(z,x)$$

Applications:

- Hyperparameter selection
- Deep Equilibrium Models
- Neural architecture search
- Data Augmentation

Implicit differentiation

Gradient of h:

$$\nabla h(x) = \nabla_2 F(z^*(x), x) + \nabla_{21}^2 G(z^*(x), x) v^*(x)$$

with

$$v^*(x) = -[\nabla_{11}^2 G(z^*(x), x)]^{-1} \nabla_1 F(z^*(x), x)$$

Implicit differentiation

Gradient of h:

$$\nabla h(x) = \nabla_2 F(z^*(x), x) + \nabla_{21}^2 G(z^*(x), x) v^*(x)$$

with

$$v^*(x) = -[\nabla_{11}^2 G(z^*(x), x)]^{-1} \nabla_1 F(z^*(x), x)$$

Bottlenecks

- One optimization problem to solve at each iteration

Implicit differentiation

Gradient of h:

$$\nabla h(x) = \nabla_2 F(z^*(x), x) + \nabla_{21}^2 G(z^*(x), x) v^*(x)$$

with

$$v^*(x) = -[\nabla_{11}^2 G(z^*(x), x)]^{-1} \nabla_1 F(z^*(x), x)$$

Bottlenecks

- One optimization problem to solve at each iteration
- One linear system to solve at each iteration

Implicit differentiation

Gradient of h:

$$\nabla h(x) = \nabla_2 F(z^*(x), x) + \nabla_{21}^2 G(z^*(x), x) v^*(x)$$

with

$$v^*(x) = -[\nabla_{11}^2 G(z^*(x), x)]^{-1} \nabla_1 F(z^*(x), x)$$

Bottlenecks

- One optimization problem to solve at each iteration
- One linear system to solve at each iteration
- Computation of the full batch derivatives when m and n are large

General framework

Idea: Maintain the variables z, v, x that evolve at the same time following suitable directions.

General framework

Idea: Maintain the variables z, v, x that evolve at the same time following suitable directions.

- $D_z(z, v, x) = \nabla_1 G(z, x)$: gradient step towards $z^*(x)$

General framework

Idea: Maintain the variables z, v, x that evolve at the same time following suitable directions.

- $D_z(z, v, x) = \nabla_1 G(z, x)$: gradient step towards $z^*(x)$
- $D_v(z, v, x) = \nabla_{11}^2 G(z, x) v + \nabla_1 F(z, x)$: gradient step towards $v^*(x)$

General framework

Idea: Maintain the variables z, v, x that evolve at the same time following suitable directions.

- $D_z(z, v, x) = \nabla_1 G(z, x)$: gradient step towards $z^*(x)$
- $D_v(z, v, x) = \nabla_{11}^2 G(z, x)v + \nabla_1 F(z, x)$: gradient step towards $v^*(x)$
- $D_x(z, v, x) = \nabla_{21}^2 G(z, x)v + \nabla_2 F(z, x)$: gradient step towards x^*

General framework

Idea: Maintain the variables z, v, x that evolve at the same time following suitable directions.

- $D_z(z, v, x) = \nabla_1 G(z, x)$: gradient step towards $z^*(x)$
- $D_v(z, v, x) = \nabla_{11}^2 G(z, x)v + \nabla_1 F(z, x)$: gradient step towards $v^*(x)$
- $D_x(z, v, x) = \nabla_{21}^2 G(z, x)v + \nabla_2 F(z, x)$: gradient step towards x^*

for $t = 0, \ldots, T-1$ **do**

Update z: $z^{t+1} = z^t - \rho^t D_z^t$,

Update v: $v^{t+1} = v^t - \rho^t D_v^t$,

Update x: $x^{t+1} = x^t - \gamma^t D_x^t$,

where D_z^t, D_v^t and D_x^t are unbiased estimators of $D_z(z^t, v^t, x^t), D_v(z^t, v^t, v^t)$ and $D_x(z^t, v^t, x^t)$.

SOBA (StOchastic Bilevel Algorithms)

Pick $i \in \{1, \ldots, n\}$ and $j \in \{1, \ldots, m\}$ and take

$$
\begin{aligned}
D_z^t &= \nabla_1 G_i(z^t, x^t) \\
D_v^t &= \nabla_{11}^2 G_i(z^t, x^t) v^t + \nabla_1 F_j(z^t, x^t) \\
D_x^t &= \nabla_{21}^2 G_i(z^t, x^t) v^t + \nabla_2 F_j(z^t, x^t)
\end{aligned}
$$

SOBA (StOchastic Bilevel Algorithms)

Pick $i \in \{1, \ldots, n\}$ and $j \in \{1, \ldots, m\}$ and take

$$
\begin{aligned}
D_z^t &= \nabla_1 G_i(z^t, x^t) \\
D_v^t &= \nabla_{11}^2 G_i(z^t, x^t) v^t + \nabla_1 F_j(z^t, x^t) \\
D_x^t &= \nabla_{21}^2 G_i(z^t, x^t) v^t + \nabla_2 F_j(z^t, x^t)
\end{aligned}
$$

Convergence rate

$$
\frac{1}{T} \sum_{t=1}^{T} \mathbb{E}[\|\nabla h(x^t)\|^2] = \mathcal{O}(T^{-\frac{1}{2}})
$$

Same convergence rate as SGD for non-convex single level problems![a]

[a] Saeed Ghadimi and Guanghui Lan. Stochastic first-and zeroth-order methods for nonconvex stochastic programming, *SIAM Journal on Optimization*, 2013

SABA (Stochastic Average Bilevel Algorithm)

Use SAGA-like variance reduction to estimate

$$\nabla_1 G(z^t, x^t), \quad \nabla_1 F(z^t, x^t), \quad \nabla_2 F(z^t, x^t)$$
$$\nabla_{12}^2 G(z^t, x^t) v^t, \quad \nabla_{11}^2 G(z^t, x^t) v^t$$

and plug the results in D_z^t, D_v^t and D_x^t.

SABA (Stochastic Average Bilevel Algorithm)

Use SAGA-like variance reduction to estimate

$$\nabla_1 G(z^t, x^t), \quad \nabla_1 F(z^t, x^t), \quad \nabla_2 F(z^t, x^t)$$
$$\nabla_{12}^2 G(z^t, x^t) v^t, \quad \nabla_{11}^2 G(z^t, x^t) v^t$$

and plug the results in D_z^t, D_v^t and D_x^t.

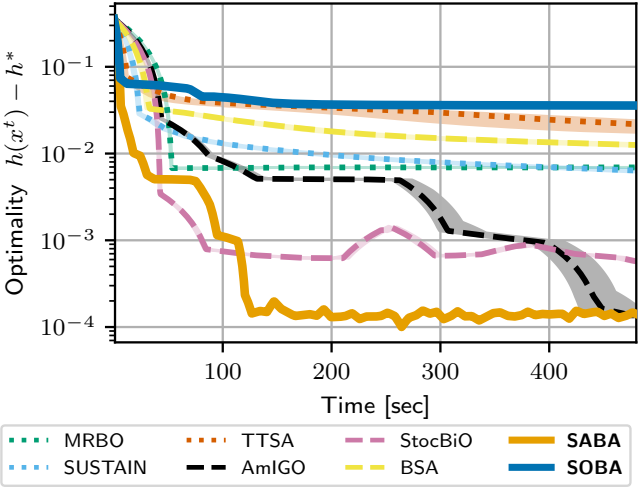
Convergence rate

$$\frac{1}{T} \sum_{t=1}^{T} \mathbb{E}[\|\nabla h(x^t)\|^2] = \mathcal{O}((n+m)^{\frac{2}{3}} T^{-1})$$

Same convergence rate as SAGA for non-convex single level problems![a]

[a] S. J. Reddi, S. Sra, B. Póczos and A. Smola, Fast incremental method for smooth nonconvex optimization, In *2016 IEEE 55th Conference on Decision and Control (CDC)*, 2016

Hyperparameter selection on ℓ^2 regularized logistic regression



Take home message

- It is possible to adapt any kind of single level stochastic optimizer to our framework.
- SOBA and SABA achieve the same sample complexity as their single level counterparts.

`https://arxiv.org/abs/2201.13409`